

ALULINK

Connecting ALU Students to Verified ALU-Run Startups

ALULink

A Firebase-Backed Flutter Marketplace Connecting ALU
Students to Verified ALU-Run Startups

Prepared by	Ayomide Adeleye
Course / Track	Mobile Application Development
Institution	African Leadership University (ALU)
Date	July 12, 2026

1. Executive Summary

ALULink is a cross-platform Flutter application that operates a two-sided marketplace between African Leadership University students seeking real-world experience and ALU-affiliated startup founders seeking campus talent. The system is built on a single Dart codebase (compiling to Web, Android, and Windows) backed entirely by Firebase — Firebase Authentication for identity, and Cloud Firestore for every piece of persisted, real-time data. There is no custom backend server: Firestore's security rules are the sole authorization boundary between clients.

The finished product implements the complete lifecycle of a talent marketplace: Google-authenticated sign-in gated to ALU email domains, role-based onboarding for students and founders, role posting with founder-authored screening questions, a matching and application flow, a drag-and-drop Kanban pipeline for reviewing applicants, real-time in-app messaging, an interview-scheduling workflow, an offer/decline resolution step with visible outcomes on both sides, and a PDF-exportable “Experience Passport” that gives students a portable, founder-verified record of their work. Every one of these flows is backed by live Cloud Firestore reads and writes rather than mock data — a deliberate correction made mid-project after an audit found several screens still displaying fixture (sample) data instead of real records.

This report documents the system as it exists in the shipped codebase: its architecture, its data model, the reasoning behind every non-trivial engineering decision, the real defects that were found and fixed during development, and the trade-offs that remain open for future iterations. Nothing described here is aspirational — every claim is traceable to a specific file, class, or method in the project.

2. Project Background

African Leadership University runs an unusually large number of small, student-founded startups operating out of its Kigali campus — in education technology, agriculture, health, and logistics. These ventures need cheap, fast, trustworthy access to skilled labour; ALU students need real, resume-worthy experience that a lecture hall cannot provide. Historically this matching has happened informally, through WhatsApp groups and word of mouth, with no verification, no structured screening, and no durable record of what a student actually did once the gig ended.

ALULink formalizes that informal economy into a single application, modeled after the mechanics of professional talent marketplaces (LinkedIn Jobs, AngelList Talent) but scoped tightly to one campus community. The defining constraint that shapes almost every design decision in this report is trust: every account must be a verified member of the ALU community, and every unit of work experience should eventually be attributable to a specific, verified founder who can vouch for it.

3. Problem Statement

Three concrete problems motivate the system:

- Students have no single, trustworthy channel to discover real roles at ALU-affiliated startups, and no way to prove, after the fact, that the experience they list on a CV actually happened and was any good.
- Founders have no lightweight way to reach a pre-qualified, ALU-only applicant pool, screen them with role-specific questions, and manage a pipeline of applicants without falling back to spreadsheets or chat threads.
- Neither side has a shared source of truth for the state of an application — has it been seen? Is an interview scheduled? Was an offer extended? — so status updates get lost in informal channels.

ALULink's job is to be that shared source of truth: one Firestore document per application, one real-time listener per screen that cares about it, and one authorization rule per collection that guarantees each side only sees what it is entitled to see.

4. Objectives

- Build a single Flutter codebase that serves two distinct user roles (student, founder) behind one authentication gate, with role-appropriate navigation shells.
- Replace every hand-authored mock/fixture screen with a real, Firestore-backed equivalent, so a fresh account starts genuinely empty and only ever shows data that a real user created.
- Enforce ALU-only membership at the authentication layer, not just in the UI copy.
- Model the full hiring funnel as real state transitions — applied → reviewed → interview → offer/decline — with a visible, honest outcome on both the founder's and the student's screens.
- Give students a durable, exportable artifact (the Experience Passport PDF) that survives beyond the app.
- Keep the security model server-enforced: every Firestore collection carries a rule that checks request.auth against the document's ownership fields, so the client UI is never the only gatekeeper.

5. Scope

5.1 In Scope

- Google Sign-In authentication restricted to @alustudent.com / @alueducation.com accounts.
- Student flows: onboarding/skills setup, Explore/Home discovery with filtering, applying to a role (with dynamic, founder-authored screening questions), tracking applications, saved roles, real-time chat, notifications, and the Experience Passport (in-app and PDF export).

- Founder flows: startup verification (with a real uploaded logo and affiliation-proof attachment), posting a role (with a screening-question builder supporting short-answer and single-choice questions), a drag-and-drop Kanban pipeline, applicant detail with match scoring, interview scheduling, offer/decline resolution, and analytics (funnel + top applicant skills).
- A shared design system (colors, type scale, shape tokens) and a reusable widget library used by every screen in both roles.
- Cloud Firestore as the only data store, with hand-authored security rules per collection.

5.2 Out of Scope (Explicitly, for This Iteration)

- Email/password authentication — the sign-in screen renders the fields from the original design mockup but only the Google button is wired to real Firebase Auth, matching the specific requirement given for this phase.
- Firebase Cloud Storage, Cloud Messaging, and Cloud Functions — not provisioned; their absence directly shapes several decisions documented in § 32 (Engineering Trade-offs) and § 38 (Limitations).
- Automated test coverage beyond the default Flutter widget-test scaffold.
- Multi-founder teams per startup, payments, and background-check integrations.

6. Functional Requirements

ID	Requirement	Primary Screen(s) / Component
FR-1	User signs in with a Google account restricted to ALU email domains	SignInScreen, AuthService
FR-2	Student completes a profile (skills, hours/week, interests) before entering the app	StudentSetupScreen
FR-3	Founder submits startup identity (name, logo, category, affiliation proof) before posting	StartupVerificationScreen
FR-4	Founder can create, edit (via delete + repost), and delete a role posting	PostRoleScreen, PostingsScreen
FR-5	Founder can attach 0–3 custom screening questions (short answer or single choice) to a posting	PostRoleScreen
FR-6	Student can browse/filter live postings and view full role details with a computed match %	ExploreScreen, OpportunityDetailsScreen
FR-7	Student can save/unsave a role for later	Home/Explore/Saved screens
FR-8	Student can apply to a role, answering the founder's real screening questions	ApplyFlowScreen

FR-9	Founder can review applicants in a drag-and-drop Kanban pipeline across stages	PipelineScreen
FR-10	Founder can message an applicant and schedule an interview with proposed time slots	StartupChatThreadScreen, SchedulerScreen
FR-11	Founder can extend an offer or decline an applicant, each with a confirmation step	ApplicantDetailScreen
FR-12	Student sees a real, non-generic outcome banner once a decision is made	ApplicationDetailScreen
FR-13	Either party can chat in real time once a relationship exists between them	ChatThreadScreen family
FR-14	Student can export a PDF “Experience Passport” of verified roles and endorsed skills	PassportScreen, passport_pdf.dart
FR-15	Either role can switch workspaces from Profile, gated on having actually set up the other role	ProfileScreen, StartupProfileScreen

7. Non-Functional Requirements

Category	Requirement	How It Is Met
Security	No client can read or write data belonging to another user	Per-collection Firestore rules keyed on request.auth.uid (§ 21)
Consistency	Every list screen reflects backend changes without a manual refresh	StreamBuilder over Firestore .snapshots() everywhere (§ 13)
Portability	One codebase runs on Web, Android, and Windows	Pure Flutter/Dart, no platform-channel-specific business logic
Usability	A visually consistent, branded UI across 58 screens	Central design system in theme.dart + shared widget library (§ 18)
Resilience	A screen with zero data must never crash or look broken	EmptyStateView used consistently; defensive .isEmpty guards after the fixture-data audit (§ 24)
Data integrity	Large uploads must not corrupt or overflow a Firestore document	Client-side image decode/resize before persisting a base64 logo (§ 20)
Maintainability	A new screen should be able to reuse existing UI primitives	18 shared widgets in lib/widgets/ used across all 58 screens (§ 19)

8. Technology Stack

Layer	Technology	Why It Was Chosen
Client framework	Flutter 3.44 / Dart 3.12	Single codebase for Web + Android + Windows; the mockup was already built frame-by-frame for a Flutter widget tree
Backend	Firebase (Firestore + Authentication)	Zero server to provision or operate; real-time listeners map directly onto Flutter's StreamBuilder
Auth provider	Google Sign-In (firebase_auth, google_sign_in)	ALU issues Google Workspace accounts, so domain-gating a Google sign-in is sufficient identity verification
Fonts / Icons	google_fonts (Plus Jakarta Sans), material_symbols_icons	Matches the hand-designed mockup's typography and iconography exactly
Media	image_picker, file_picker, image (pure-Dart resize)	Real logo/photo/document selection without needing Firebase Storage (\$ 20)
Documents	pdf, printing	Generates the Experience Passport as a real, shareable, offline artifact
Motion	confetti, shimmer	Celebratory success feedback and skeleton loading states, not purely decorative
Tooling	flutter_lints, flutter test	Static analysis gate (\$ 25) and the default widget-test harness

9. Project Folder Structure

The codebase is organized by feature area under lib/, with a strict layering discipline enforced by convention rather than a build tool: screens import widgets and services, widgets never import screens, and models import neither Flutter widgets nor Firestore types directly.

```
lib/
├── main.dart                Firebase.initializeApp() then runApp()
├── firebase_options.dart    generated by FlutterFire CLI (per-platform config)
├── theme.dart              AppColors / AppText / AppShape design tokens + ThemeData
├── data/
│   ├── models.dart        domain model classes (StudentProfile, Opportunity, ...)
│   ├── fixtures.dart      single-user local StudentProfile holder (legacy demo data removed)
│   └── skills.dart / student_options.dart  static picker option lists
├── services/
│   ├── auth_service.dart   Firebase Auth + ALU domain gate (singleton)
│   └── marketplace_repository.dart  every Firestore read/write (singleton)
```

└─ passport_pdf.dart	builds the Experience Passport PDF document
└─ widgets/	18 shared, stateless UI building blocks
└─ screens/	
└─ a_auth/	splash → onboarding → role select → sign in → setup/verification
└─ b_student/	Home, Explore, Apply, Applications, Saved, Chat, Passport, Profile
└─ c_startup/	Dashboard, Postings, Pipeline, Applicant Detail, Analytics, Profile
└─ d_states/	dev-reference screens for loading/empty/offline states
└─ dev/	screen_gallery.dart – a hidden QA index of every screen
└─ library/	component_sheet_screen.dart – a hidden design-system reference

58 screen files in total, split roughly 19 / 22 / 9 / 3 / 1 / 1 across the six folders above — a ratio that reflects how much richer the founder-side pipeline/analytics tooling is compared to the student's more linear browse-and-apply flow.

10. System Architecture

ALULink follows a client-heavy Backend-as-a-Service (BaaS) architecture: there is no application server written for this project. The Flutter client authenticates directly against Firebase Authentication and reads/writes directly against Cloud Firestore over the official SDKs. The only “backend logic” that exists is declarative — Firestore Security Rules — rather than imperative server code.

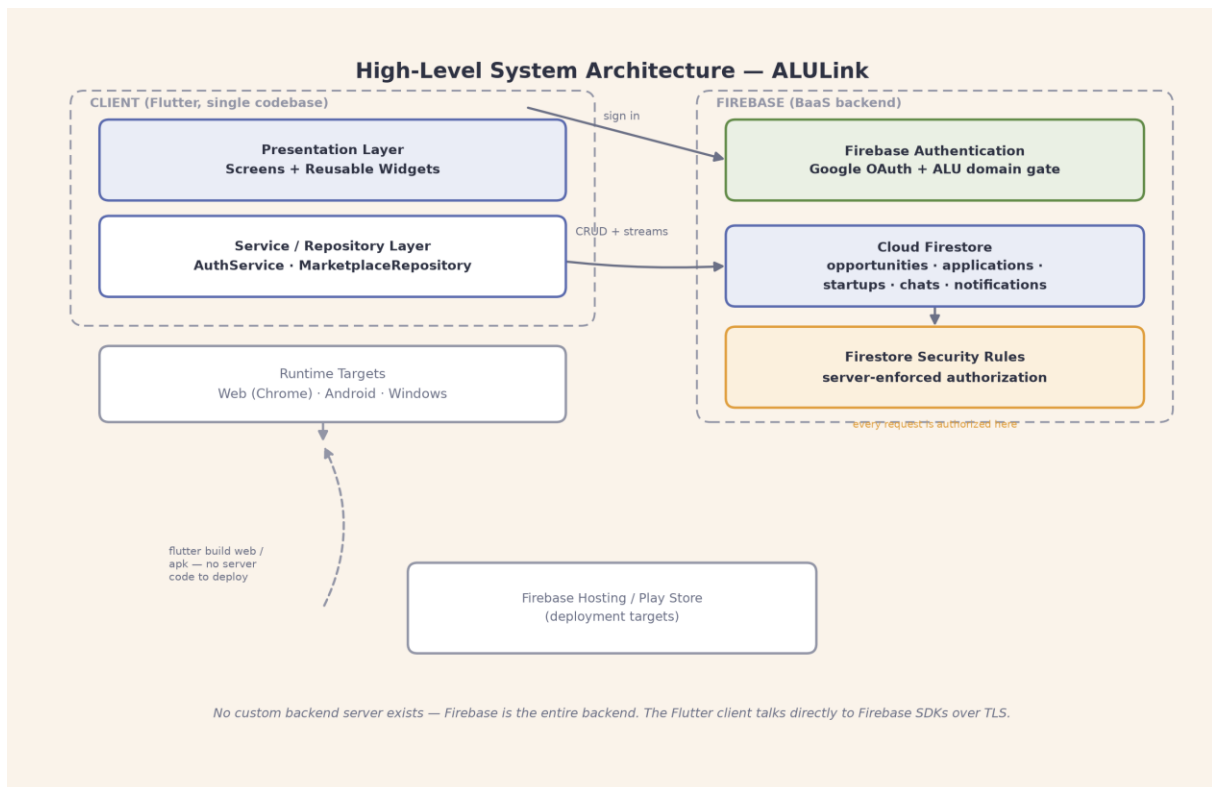


Figure 1. High-level system architecture: a single Flutter client talks directly to Firebase Authentication and Cloud Firestore, gated by server-enforced security rules.

This is a deliberate choice, not a limitation born of time pressure. For a single-campus marketplace with moderate write volume and no requirement for server-side business logic (payment processing, third-party webhooks, scheduled batch jobs), a BaaS backend removes an entire category of operational risk: there is no server to patch, scale, or take down for maintenance. The cost of this choice — logic that would normally live in a trusted server now has to be either duplicated on the client or expressed as a security rule — is discussed explicitly in § 20.

11. Architectural Patterns Used



Figure 2. The six-layer client architecture. Each layer only calls downward.

11.1 Repository Pattern

MarketplaceRepository (lib/services/marketplace_repository.dart) is the single Repository [3] standing between every screen and Cloud Firestore. No screen ever imports cloud_firestore directly or constructs a query by hand — it calls a method like watchMyPostings(), apply(), or declineApplicant() and gets back either a Future (for one-shot writes) or a Stream<List<T>> (for anything a screen needs to keep watching). This buys two things: (1) every Firestore document shape is translated to/from a typed Dart model in exactly one place (the _xFromDoc() family of private methods), so a schema change touches

one file, and (2) match-scoring, notification side-effects, and denormalization logic live next to the writes they accompany instead of being scattered across the UI layer.

11.2 Singleton Services

AuthService and MarketplaceRepository are both classic singletons (private constructor + static instance field). This mirrors how Firebase's own SDKs are structured (FirebaseAuth.instance, FirebaseFirestore.instance) and keeps the object graph trivial to reason about — every screen reaches the same repository instance, so a write from one screen is immediately visible to a stream subscribed elsewhere. The explicit trade-off this makes against dependency injection is discussed in § 20.

11.3 Shell + IndexedStack Navigation

Both StudentShell and StartupShell use an IndexedStack behind a bottom navigation bar rather than separate routes for each tab. This preserves each tab's scroll position and StreamBuilder subscriptions when switching tabs — a real usability win, since re-subscribing to Firestore on every tap would reset scroll position and briefly show a loading spinner every time.

11.4 Denormalized Documents Over Joins

Firestore has no server-side joins. Rather than storing a bare opportunityId on an Application document and fetching the parent Opportunity on every read, MarketplaceRepository.apply() copies the fields a list screen actually needs (opportunityTitle, startupName, startupGradientSeed, requiredSkills, ...) directly onto the application document at write time. This is the standard NoSQL trade-off: pay a small, predictable cost once at write time, in exchange for every list screen (Applicants, Applications, Postings) rendering from a single query instead of an N+1 fan-out of reads.

12. State Management Architecture

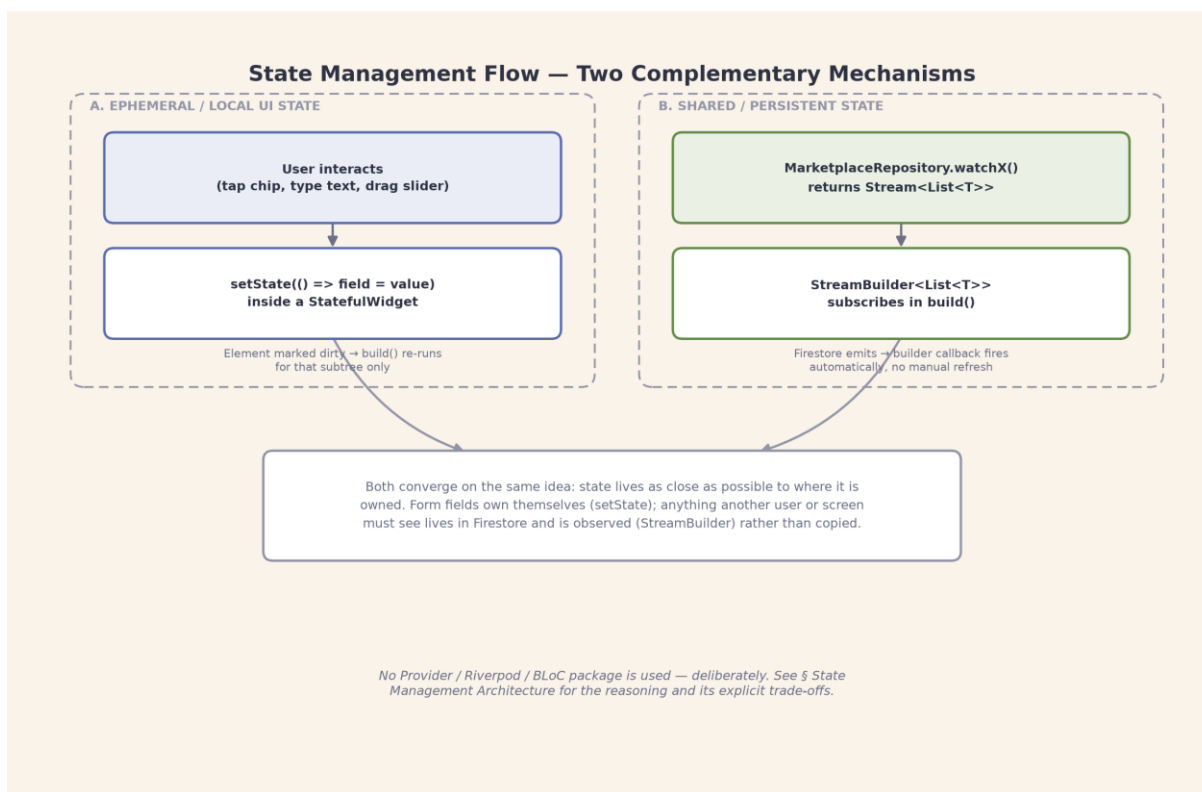


Figure 3. The two state-management mechanisms used throughout the app, and where each applies.

ALULink intentionally does not depend on Provider, Riverpod, BLoC, or GetX. State is managed with exactly two built-in Flutter mechanisms, chosen based on who owns the state:

- Ephemeral, single-screen state (a text field's contents, which chip is selected, which onboarding slide is showing) lives in a StatefulWidget's own fields and is mutated with setState(). No other screen needs to know about it, so there is no reason to hoist it into a shared store.
- Shared, cross-screen, backend-owned state (opportunities, applications, chat messages, notifications) is never copied into a client-side store at all — it is observed directly from Cloud Firestore via StreamBuilder<List<T>>, so Firestore itself is the single source of truth and every subscribed screen re-renders automatically the instant the backend changes.

This is a conscious architectural bet: for an app whose defining feature is that two different users (a founder and a student) need to see the same fact change in real time, a Firestore stream already is the reactive state layer a state-management package would otherwise be built to simulate. Introducing a client-side store on top would mean synchronizing two sources of truth instead of one. The cost of this bet — every screen depends on the two singletons directly, with no substitution point for tests — is one of the explicit trade-offs recorded in § 20.

13. Firebase Architecture

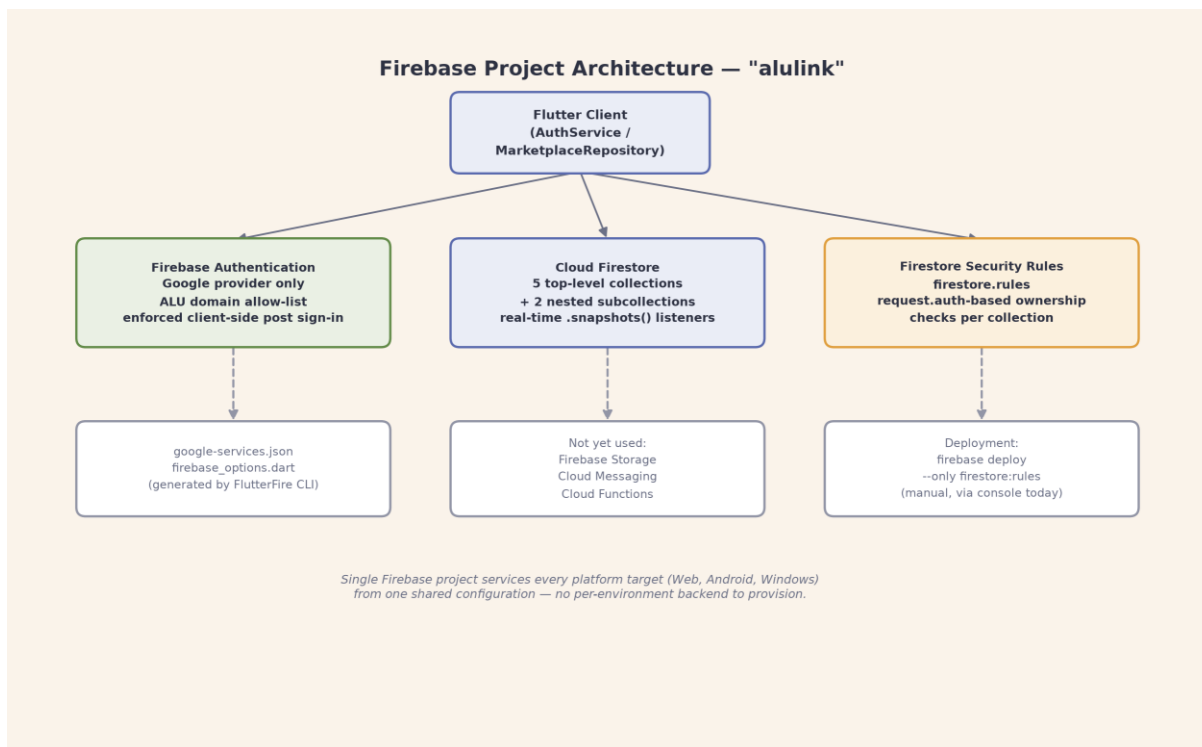


Figure 4. The single Firebase project ("alulink") and the services it provides.

One Firebase project, alulink, backs every platform target. `lib/firebase_options.dart` (generated once by the FlutterFire CLI) and `android/app/google-services.json` carry the project's API keys and app IDs; neither is a secret in the traditional sense — Firebase's real security boundary is the rules layer, not the visibility of these config values. Two Firebase products are actually used: Firebase Authentication (Google provider only) and Cloud Firestore. Firebase Storage, Cloud Messaging, and Cloud Functions are provisioned nowhere in this project; every place their absence is felt is called out explicitly rather than silently worked around (§ 32, § 38).

14. Firestore Database Design

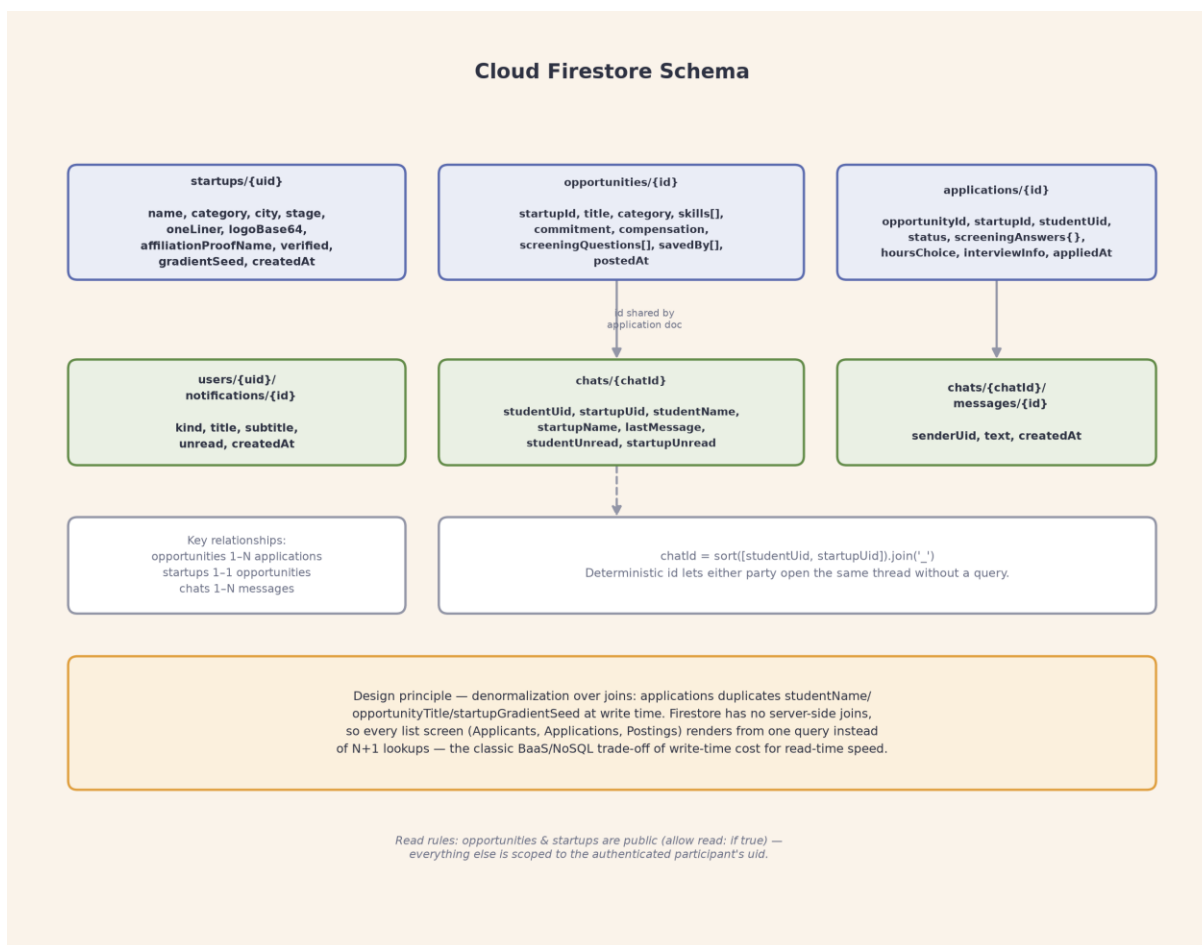


Figure 5. Cloud Firestore schema — five top-level collections and two nested subcollections.

Firestore is a document database, not a relational one, so the schema is designed around the read patterns each screen actually needs rather than around normalized entities. The five top-level collections are:

Collection	Key Fields	Written By	Read By
startups/{uid}	name, category, city, stage, logoBase64, affiliationProofName, verified, gradientSeed	saveStartupProfile()	Dashboard, Postings, chat headers, public profile
opportunities/{id}	startupId, title, skills[], compensation, screeningQuestions[], savedBy[]	postOpportunity()	Home, Explore, Postings, Saved
applications/{id}	opportunityId, startupId, studentUid, status, screeningAnswers{}	apply(), updateStage(),	Applications, Pipeline, Applicant Detail,

	interviewInfo	declineApplicant()	Analytics
<code>users/{uid}/notifications/{id}</code>	kind, title, subtitle, unread	<code>_notify()</code> (internal helper)	Notifications screen (both roles)
<code>chats/{chatId} + /messages/{id}</code>	studentUid, startupUid, lastMessage, studentUnread, startupUnread	<code>getOrCreateChat...()</code> , <code>sendMessage()</code>	Messages list + Chat Thread (both roles)

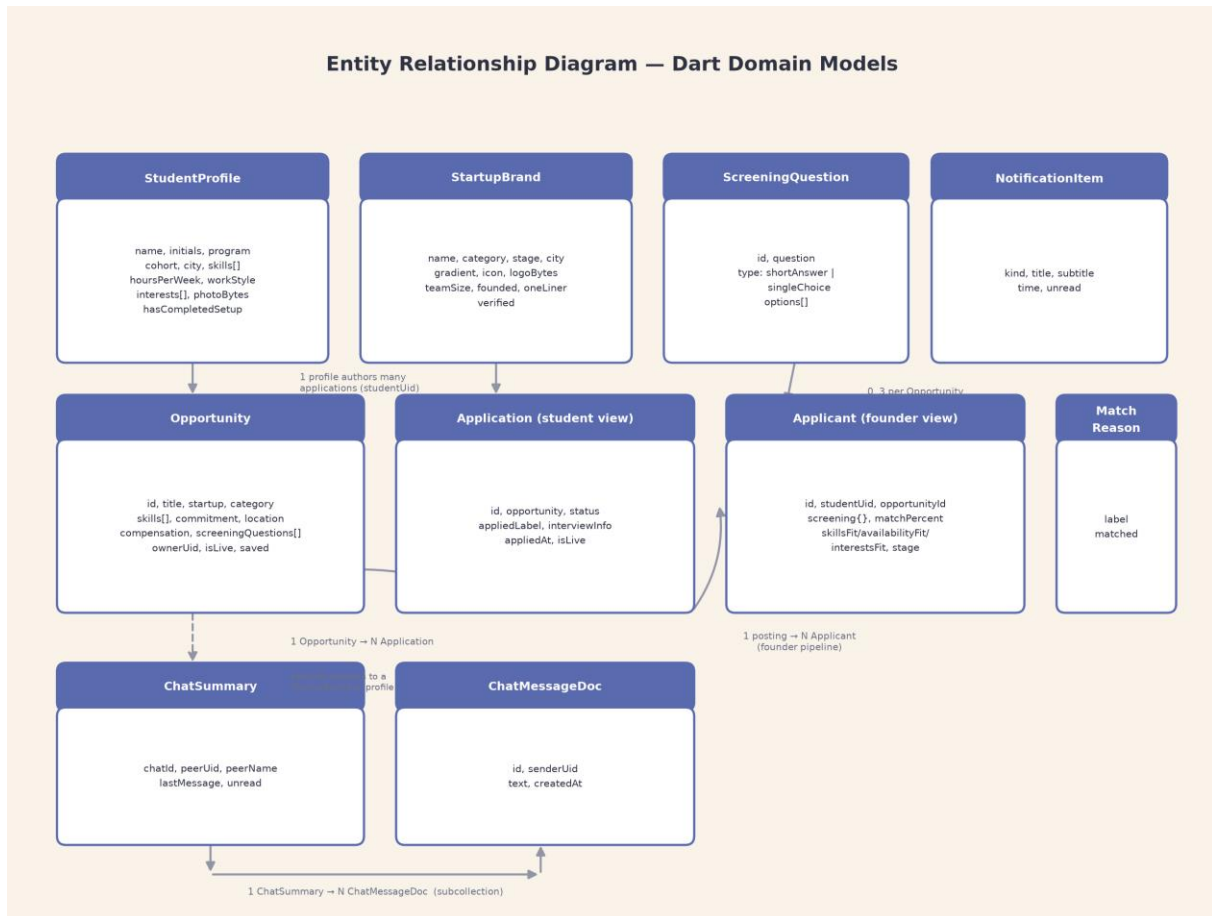


Figure 6. Entity relationship diagram of the Dart domain models that wrap each Firestore document.

14.1 The Chat ID Derivation

Rather than querying for an existing thread between two users, a chat's document ID is computed deterministically: `chatId = sort([studentUid, startupUid]).join('_')`. Either party can independently compute the same ID and call `.doc(chatId).set(merge: true)` — the first caller creates the thread, the second one merges into it, and no query or Cloud Function is needed to “find or create” a conversation.

14.2 Screening Questions as an Embedded Array

A posting's screeningQuestions field is an array of small maps ({type, question, options}) embedded directly on the opportunity document rather than a separate subcollection. Screening questions are always read as a whole alongside their parent posting and capped at three by product design, so there is no read pattern that benefits from a subcollection — embedding avoids an extra round trip for data that is never queried independently.

Design principle: every collection is shaped by "what does a screen need in one query," not by textbook third-normal-form. This is standard Firestore practice — see the official modeling guide [2].

15. Authentication Workflow

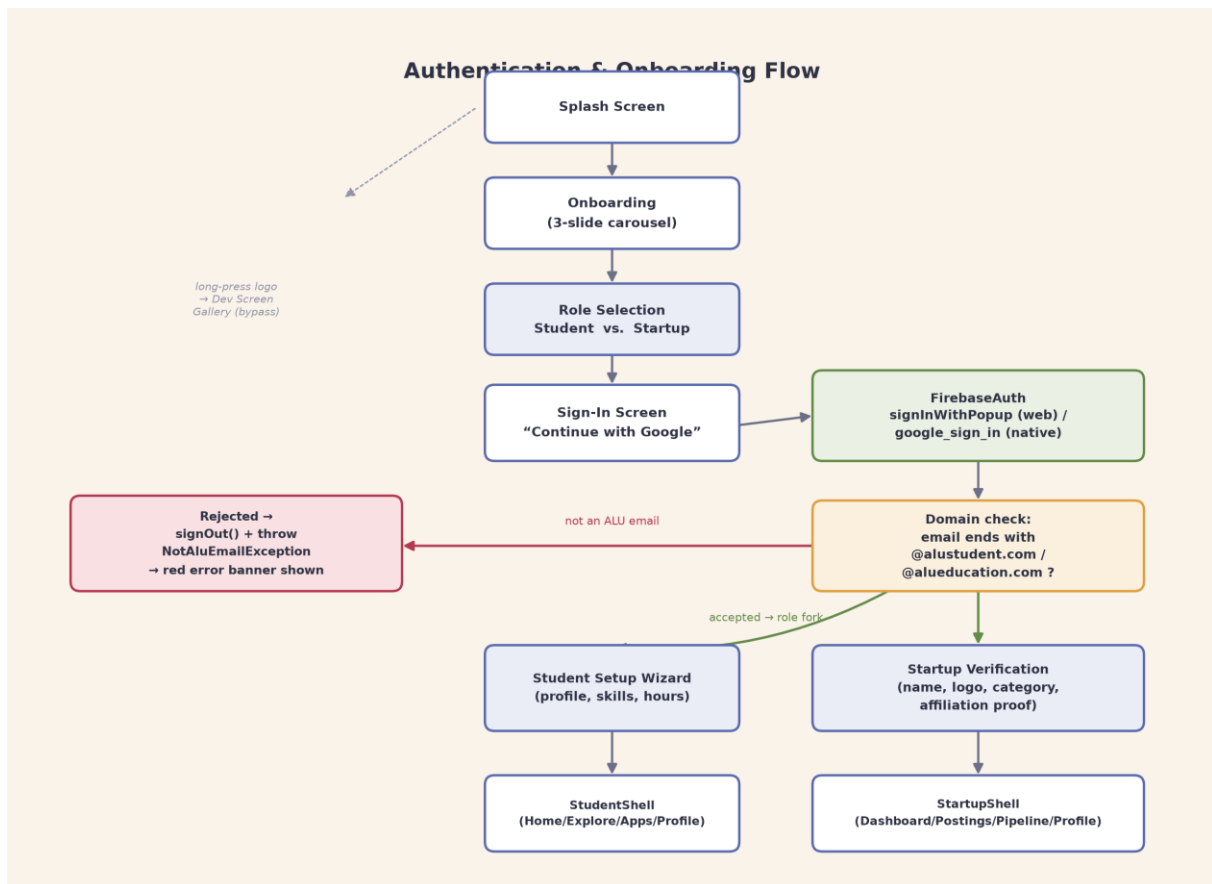


Figure 7. End-to-end authentication and onboarding flow, including the ALU-domain rejection path.

AuthService.signInWithGoogle() branches on platform: on Web it calls FirebaseAuth.instance.signInWithPopup(GoogleAuthProvider()); on Android/iOS it drives the google_sign_in picker and exchanges the resulting tokens for a Firebase credential. Both paths converge on the same post-sign-in check: isAluEmail() verifies the signed-in account's email ends in

@alustudent.com or @alueducation.com. If it does not, the code calls `signOut()` immediately and throws a typed `NotAluEmailException`, which `SignInScreen` catches to show the user's actual rejected email address in a red banner rather than a generic failure message.

Engineering note — *A real bug fixed during development:*

`GoogleSignIn()` must never be constructed eagerly as a field. On Flutter Web, simply instantiating that class outside of a lazy getter triggers the `google_sign_in_web` plugin's initialization path, which asserts and crashes because that path expects a web OAuth client ID that this project's web sign-in flow (`signInWithPopup`) never needs. The fix — a lazy getter (`GoogleSignIn get _googleSignIn => _googleSignInInstance ??= GoogleSignIn(...)`) — is preserved with an explicit code comment warning future maintainers not to revert it.

Role is not stored anywhere in Firestore or Firebase Auth — it is inferred at runtime from which of two documents exists for the signed-in uid: a `startups/{uid}` document (founder) or a completed local `StudentProfile` with `hasCompletedSetup=true` (student). This is why “Switch workspace” on both Profile screens has to call `getMyStartupOnce()` / check `hasCompletedSetup` before navigating, rather than simply flipping a stored role flag (§ 17 covers the product reasoning).

16. CRUD Workflow

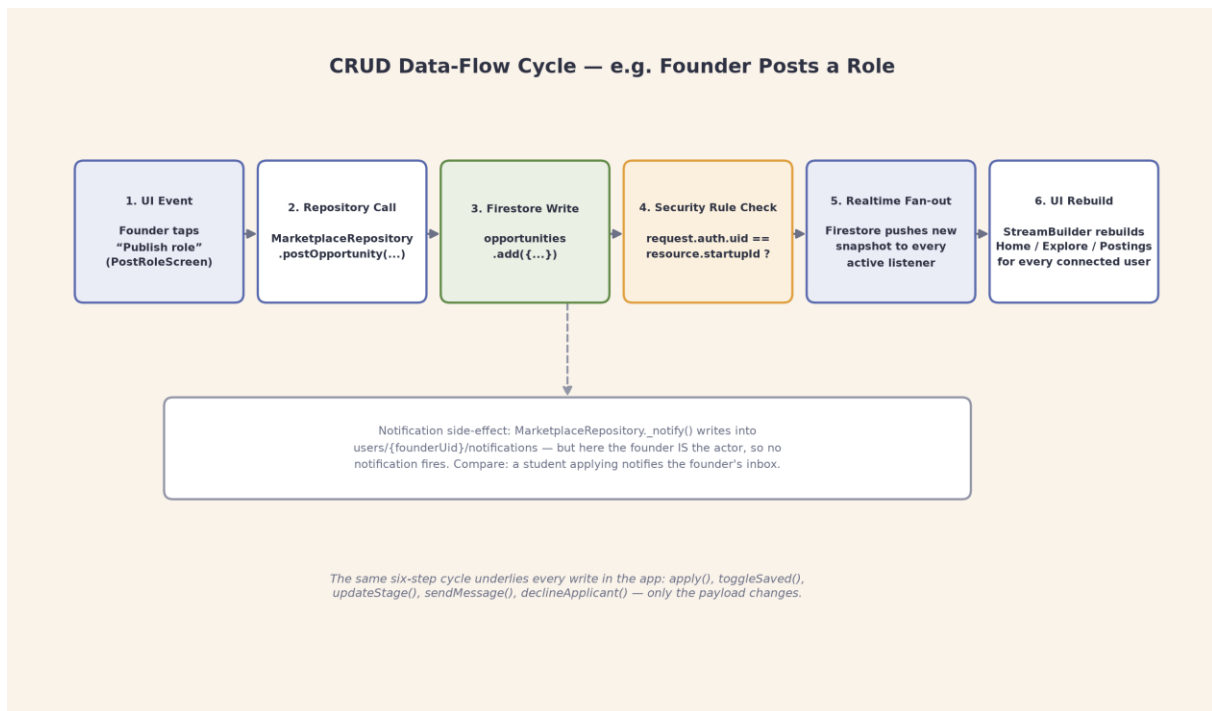


Figure 8. The six-step cycle every write in the app follows, illustrated for a founder publishing a role.

Every mutation in MarketplaceRepository follows the same shape: a screen calls a typed method, that method issues one Firestore write (.add(), .set(merge:), or .update()), Firestore's security rules evaluate the write against request.auth before committing it, and every open .snapshots() listener anywhere in the app that matches the affected query receives the new state automatically. No screen ever manually refetches — the six operations below are representative of the full CRUD surface:

Operation	Method	Notes
Create	postOpportunity(), apply(), sendMessage()	Each also fires a matching notification via the internal _notify() helper where appropriate
Read (live)	watchMyPostings(), watchApplicantsForMyPostings(), watchMessages()	Return Stream<List<T>>, consumed by StreamBuilder
Read (one-shot)	getMyStartupOnce(), getStartupByUid()	Used for gating logic where a stream would be the wrong tool (e.g. deciding whether to show a dialog)
Update	updateStage(), updateStatus(), toggleSaved(), declineApplicant()	Partial .update() calls — never a full document overwrite
Delete	deleteOpportunity()	Founder-only, gated by firestore.rules on the document's startupId field

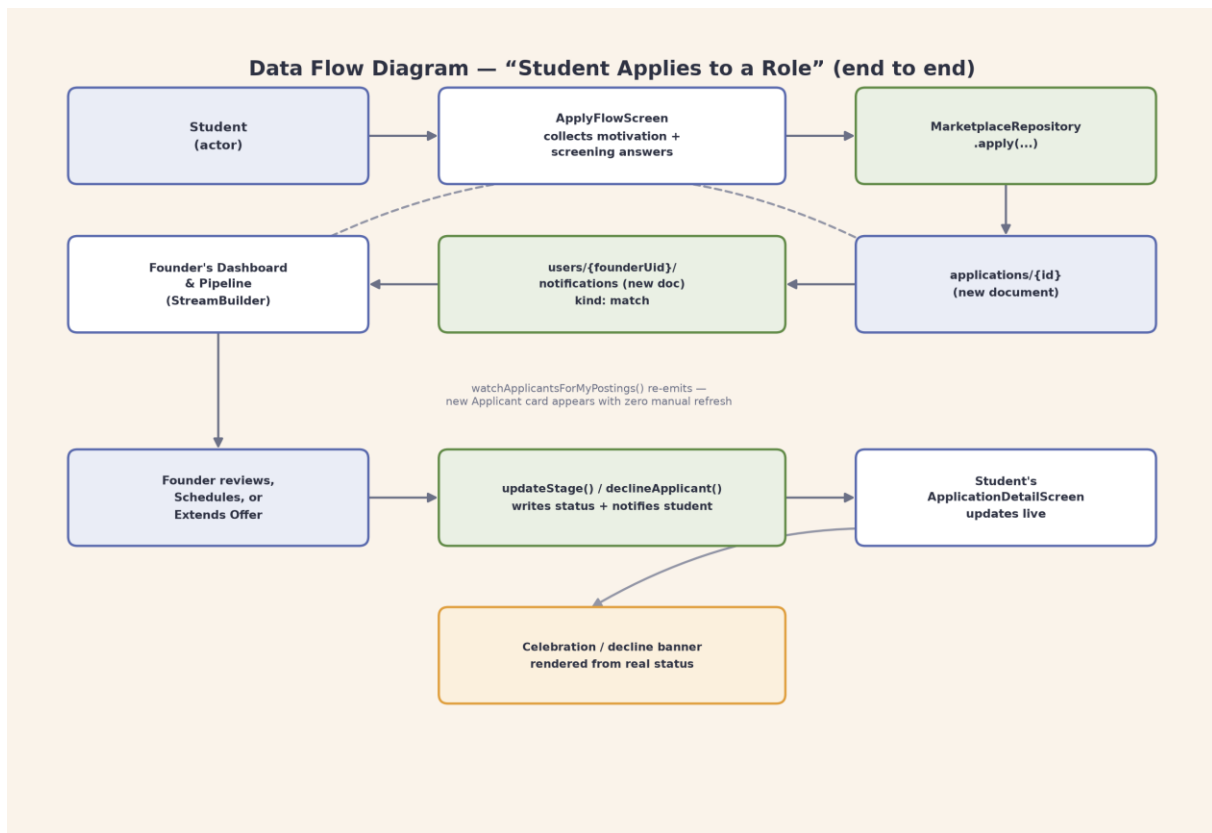


Figure 9. Data flow for a complete, concrete example — a student applying to a role, through to the founder's decision and the student seeing a real outcome.

Tracing one concrete request end to end makes the CRUD table above tangible: `ApplyFlowScreen` collects the student's motivation note and screening answers and calls `apply()`, which writes one applications document and one notification document in the same method. That new application document is what `watchApplicantsForMyPostings()` on the founder's Pipeline is already subscribed to, so the new applicant card appears with no action from the founder beyond having the screen open. When the founder later calls `updateStage()` or `declineApplicant()`, the same document updates, and the student's `ApplicationDetailScreen` — subscribed to `watchMyApplications()` — renders the real, resolved outcome banner described in § 20.2.

17. Navigation Flow

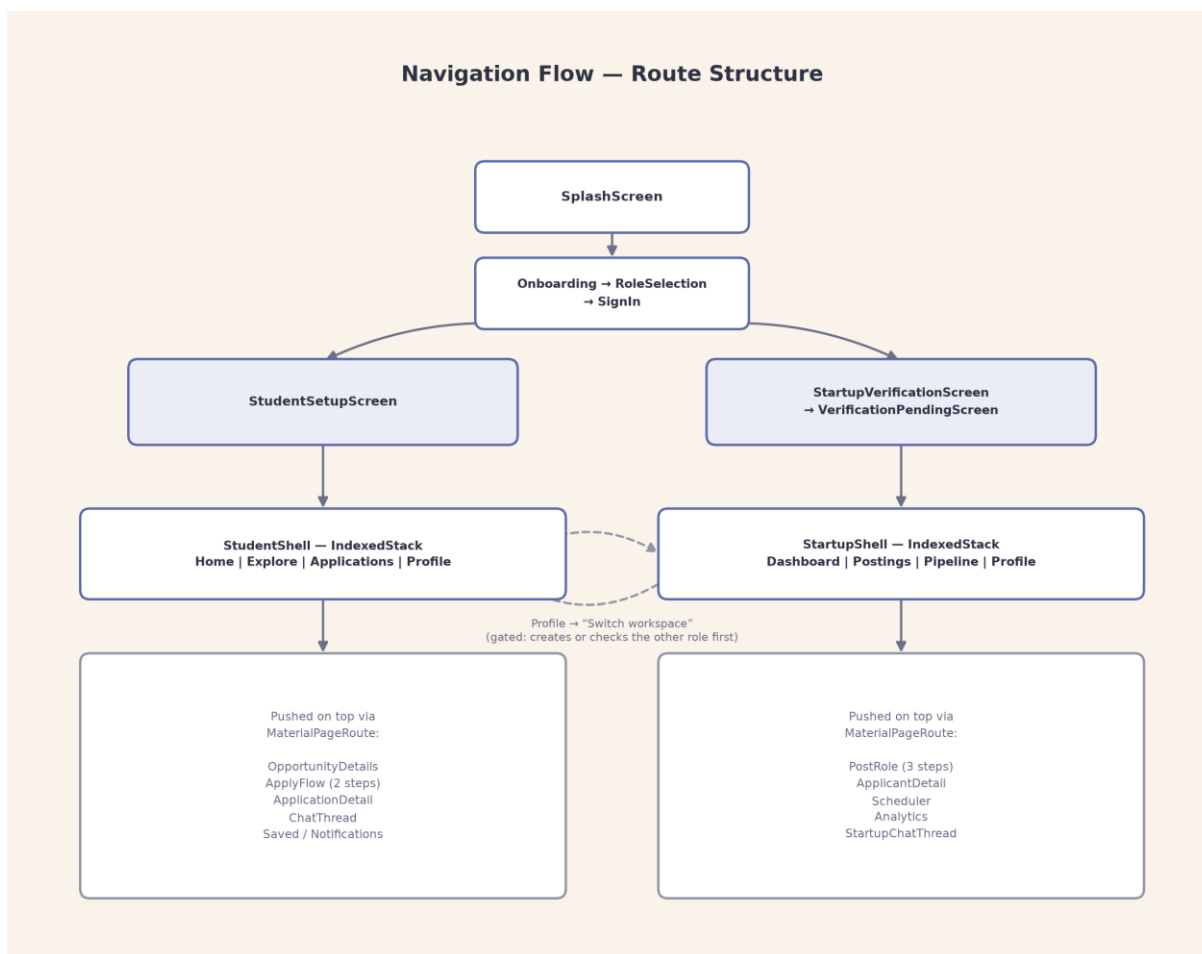


Figure 10. Route structure: an auth stack feeding into one of two role-scoped shells.

Navigation uses Flutter's imperative Navigator API (`Navigator.of(context).push(MaterialPageRoute(...))`) throughout rather than a declarative router package. Given that ALULink has exactly two static bottom-nav shells and a modest number of push-on-top detail screens, a declarative router (`go_router` is present in `pubspec.yaml` but currently only used for the two named top-level routes in `main.dart`) would add indirection without solving a problem this app actually has — deep-linking and URL-based state restoration are not requirements for a mobile-first marketplace used behind authentication.

The one genuinely non-trivial navigation decision is the “Switch workspace” action available from both Profile screens. Rather than immediately pushing the other role's shell, it always checks first: the student-to-founder direction calls `MarketplaceRepository.getMyStartupOnce()` and shows a confirmation dialog offering to start startup verification if no startup exists yet; the founder-to-student direction checks the local `hasCompletedSetup` flag and redirects to the setup wizard instead of Home if the student profile was never completed. Both guards exist for the same reason: without them, a user could be dropped into a shell whose screens assume data that was never actually created.

18. User Journey

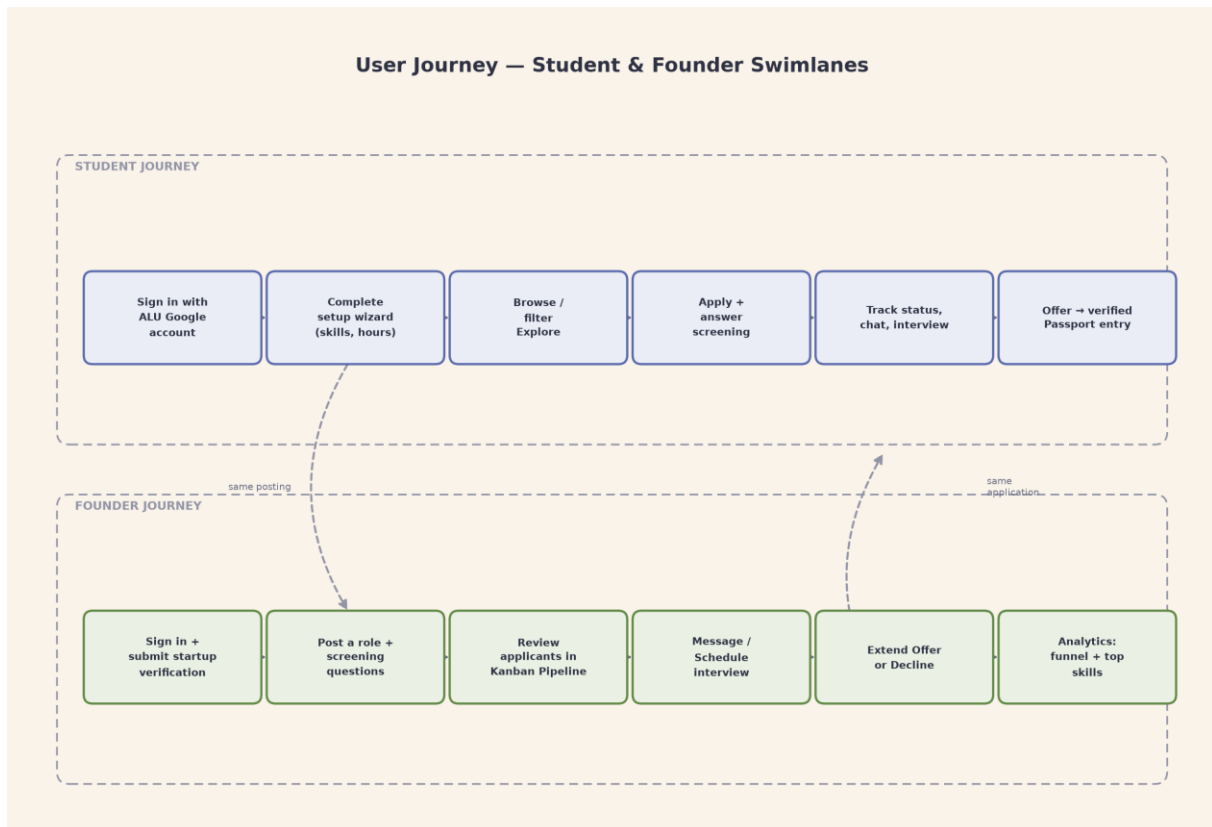


Figure 11. Parallel student and founder journeys, converging on the same postings and applications.

The two journeys are designed to interlock at exactly two points: a posting a founder creates is the same document a student applies against, and a decision a founder makes on an applicant is the same document a student sees resolve on their own Application Detail screen. Every other step in each journey is independent, which is what allows the two navigation shells (§ 17) to be built, tested, and reasoned about almost entirely separately.

19. UI/UX Design Philosophy

Every screen is built against one small, centrally defined design system rather than ad hoc styling: AppColors, AppText, and AppShape (lib/theme.dart) fix the palette, the Plus Jakarta Sans type scale, and the corner-radius/spacing/elevation tokens once, and every one of the 58 screens references those tokens instead of hard-coded values. The palette itself is intentionally warm and campus-appropriate rather than corporate: a cream background (#FAF3EA), periwinkle interactive color (#5A6AAE), and a crimson brand accent (#B33951) reserved specifically for high-commitment actions — Apply Now, Post a role, Get started — so the one color that means "this is a big decision" never gets diluted by overuse.

19.1 Status Communication Never Relies on Color Alone

StatusBadge (lib/widgets/badges.dart) always pairs a tinted background with an explicit text label — "Interview", "Accepted", "Declined" — rather than a bare colored dot. This is both an accessibility choice (§ 20) and a trust choice: in a system whose entire value proposition is being a reliable source of truth about where an application stands, ambiguity about what a color means is a product defect, not just a visual one.

19.2 Skeletons Over Spinners for List Content

Where a screen is waiting on a first Firestore snapshot for list content, the shimmer-based SkeletonBox communicates the eventual shape of the content (a card-sized shimmering rectangle) rather than an content-agnostic spinner, so the layout does not jump once data arrives.

20. Product Design Decisions

20.1 The Founder's "Extend Offer" Button Used to Lie About What It Did

An audit of the Applicant Detail screen found a green button labelled "Shortlist" whose onPressed handler actually moved the applicant to the Offer pipeline column and set their real status to accepted — while the adjacent ✕ button, which visually reads as "reject," did nothing but pop the screen with no state change at all. This was corrected: the button is now honestly labelled "Extend Offer," gated behind a confirmation dialog naming the applicant and the role, and the ✕ now performs a real declineApplicant() call (also confirmed) that writes a declined status and notifies the student. This is documented here rather than only in a commit message because it is the clearest example in the project of a UI label and its real effect silently diverging — exactly the class of bug that erodes the trust the whole product is meant to provide.

20.2 Every Resolved Application Gets an Honest Outcome, Not a Permanently "Pending" Screen

Before correction, the student's Application Detail screen rendered its final "Decision" timeline step as permanently pending ("Usually within 2 days of the interview") regardless of whether a real decision had already been made days earlier. The screen now reads the application's real status: an accepted application shows a green "You got the offer! 🎉" banner and a completed timeline node; a declined one shows a neutral, respectful "didn't work out" banner; and the now-irrelevant "Withdraw application" action is hidden once a decision exists.

20.3 Uploads Are Compressed Client-Side Rather Than Assumed Safe

Because no Firebase Storage bucket is provisioned (§ 38), a startup's logo is stored as a base64 string directly on its Firestore document. The `image_picker` package's own resize parameters are not reliably honored on Flutter Web, so a real, unresized phone photo could exceed Firestore's 1 MB per-document limit once base64-encoded — the failure mode observed during testing was a silent permission-denied-looking write failure with no useful error message. The fix decodes the picked image with the pure-Dart image package, resizes it to a 256px-max JPEG, and rejects source files over 6 MB outright before attempting the (potentially slow, unaccelerated) decode — trading a small amount of upload-time CPU work for a hard guarantee the write will never exceed the document limit.

21. Accessibility Considerations

- Status is always conveyed by a text label alongside color (§ 19.1), not by color alone, benefiting users with color vision deficiency.
- Interactive targets follow Material's touch-target guidance — `AppShape.logoTile` (44×44) is used as the minimum tap-target size for icon buttons throughout the app.
- Google Fonts' Plus Jakarta Sans is used at a base size of 14–16px for body text with generous 1.3–1.55 line-height, rather than the smaller sizes common in dense mobile UIs.
- Semantic labelling via Flutter's built-in widget semantics is inherited for free from Material widgets (`FilledButton`, `TextField`, `Switch`) used throughout — a genuine limitation is that no custom `Semantics()` annotations have been added to the bespoke widgets in `lib/widgets/`, which is recorded as a concrete improvement in § 38 rather than glossed over.

22. Responsiveness Strategy

ALULink targets a single mobile-portrait layout width and is tested primarily via `flutter run -d chrome` at a phone-sized viewport (the desktop Android emulator was found to be too slow for this project's iteration speed, so Chrome became the primary and fastest test target, per the project's own migration notes). Within that constraint, layouts still use `Expanded`/`Flexible` with text overflow ellipsis rather than fixed pixel widths wherever a Row could plausibly overflow — a real bug caught and fixed during development was the bottom navigation bar's "Applications" label overflowing its allotted width on narrower screens, resolved by wrapping each nav item in `Expanded` with ellipsis-safe text, a pattern then reused for the Home screen's category chip row and "See all" section headers.

A true responsive (phone/tablet/desktop breakpoint) layout system is not implemented — this is an explicit, named limitation in § 38, not an oversight this report tries to hide.

23. Scalability Strategy

Cloud Firestore scales horizontally by design, and the schema in § 14 is already shaped to take advantage of that: every list screen resolves with a single, indexed, denormalized query rather than a sequence of dependent reads, so read latency does not grow with the number of related documents a screen would otherwise need to join. The two places most likely to need attention as usage grows are identified explicitly rather than assumed away:

- `watchLiveOpportunities()` currently orders by `postedAt` with no pagination — acceptable at the scale of one campus's worth of postings, but the first thing to add (via Firestore's `startAfter()` cursor) if the catalog grows past a few hundred live roles.
- The Kanban Pipeline and Applicant Detail screens merge every applicant across every posting a founder owns into one client-side list; a founder running many simultaneous postings would benefit from a per-posting query rather than post-fetch filtering, which is straightforward given the existing `opportunityId` field already stored on each application document.

Because `MarketplaceRepository` is the only code that talks to Firestore, both of these are additive changes to method bodies, not schema migrations or UI rewrites — a direct payoff of the Repository pattern discussed in § 11.1.

24. Security Considerations

The client is never trusted. Every collection in `firestore.rules` carries an explicit allow rule keyed on `request.auth`, and reads/writes fall through to a default deny if no rule matches — Firestore's rules language is deny-by-default, which is itself the single most important security fact about this project's backend.

Collection	Read Rule	Write Rule
opportunities	Public (allow read: if true)	Create/update/delete only if <code>request.auth.uid == startupId</code>
applications	Only the applying student or the owning founder	Create only as the applying student; update by either party (§ 24.1)
startups	Public (needed for the founder directory + chat profiles)	Only the founder can write their own document (<code>uid == doc id</code>)
chats + messages	Only the two named participants (<code>studentUid / startupUid</code>)	Only a named participant, and only as themselves (<code>senderUid == request.auth.uid</code>)
users/{uid}/notifications	Only the owning uid	Any signed-in user (the sender, not the recipient, is the writer)

24.1 A Real Gap Found and Closed Mid-Project

For a stretch of development, `firestore.rules` had no rule at all for the `startups` collection or the `users/{uid}/notifications` subcollection — meaning every read and write to either was silently rejected under Firestore's deny-by-default behaviour. Because `MarketplaceRepository.postOpportunity()` reads a founder's own startup document before writing a new role, the symptom was a generic "Couldn't publish" toast with the real `cloud_firestore/permission-denied` exception swallowed by a blanket catch (`_`) block. The fix was two-fold and is preserved as a durable lesson: (1) add the missing rule blocks with the correct ownership checks, and (2) replace the swallowing catch blocks with `catch (e, st) { debugPrint(...) }` so a future permission gap surfaces a diagnosable error instead of a mystery failure.

24.2 What Security Explicitly Does Not Cover

Application-domain gating (the `@alustudent.com / @alueducation.com` check in § 15) happens client-side, in Dart, after Firebase Auth has already completed the sign-in. This is intentional given Firebase's constraints — Firebase Authentication itself has no first-class "restrict to these email domains" provider setting for Google Sign-In — but it is a real, named limitation: a modified or compromised client build could skip the check. The Firestore rules layer is deliberately the true trust boundary for data access; the email-domain check is a UX gate for who gets a good onboarding experience, not the system's actual authorization mechanism.

25. Data Validation

Validation is applied at the point where bad input would actually cause harm, rather than uniformly everywhere:

- Post Role's "Continue" button on step one is disabled until both title and description are non-empty (`titleController.text.trim().isEmpty && descController.text.trim().isEmpty`).
- A screening question can only be saved once its text is non-empty, and — if it is a single-choice question — it has at least two non-empty options.
- Apply Flow requires a non-empty motivation note before advancing, and the final Submit button is disabled until every one of the founder's real screening questions has an answer (a getter, `_screeningComplete`, checks every question type-appropriately).
- A picked logo image is rejected outright above 6 MB before an expensive decode is attempted (§ 20.3), and a non-image file picked for the logo slot is caught by a null check on `img.decodeImage()` rather than allowed to throw.
- Firestore Security Rules re-assert the two most important invariants — a document's `ownerUid` must equal the authenticated caller for any create — so validation is enforced twice: once for UX (client) and once for correctness (server rule), which is deliberate rather than redundant.

26. Error Handling

Every Firestore write in a screen is wrapped in try/catch, surfacing a dark, branded toast (`showAppToast`) on failure rather than letting an unhandled exception crash the widget tree. The most important lesson from this project's error-handling history, however, is about what a catch block hides: several catch (`_`) blocks were found swallowing the actual exception entirely, so every distinct Firestore failure looked identical to the user ("Couldn't publish. Please try again.") regardless of cause. These were replaced with `catch (e, st) { debugPrint('... failed: $e\n$st'); ... }` so the real `FirebaseException` code (permission-denied, and briefly, an oversized-document write) is always visible in development, while the user still only ever sees a calm, actionable message.

27. Loading States

Every `StreamBuilder` distinguishes "still connecting" from "connected with zero results" explicitly via `snapshot.connectionState == ConnectionState.waiting`, rather than treating an empty list and an unloaded list the same way. This mattered concretely once fixture data was removed (§ 20): without the distinction, a screen would flash a "nothing here yet" empty state for a fraction of a second before real data arrived, which reads as a bug even though it resolves almost instantly. A `CircularProgressIndicator` or a `SkeletonBox` now covers that window, and `EmptyStateView` (§ 19.2) only ever renders once the stream has genuinely confirmed there is nothing to show.

28. Offline Considerations

Cloud Firestore's client SDK ships with an on-device cache and offline write queue enabled by default, so brief connectivity drops on a mobile network are tolerated transparently by the platform without any additional code in this project — a write made offline is queued and flushed automatically once connectivity returns, and a `StreamBuilder` continues to serve the last-known cached snapshot in the meantime. What is explicitly not implemented is any UI acknowledgement of offline state (no "you're offline" banner) or conflict resolution beyond Firestore's own last-write-wins default — both are named directly in § 38 as scoped-out rather than silently absent.

29. Code Organization

The folder structure in § 9 is enforced by import discipline rather than a lint rule: screens import from `data/`, `services/`, and `widgets/`; widgets import only from `data/` and `theme.dart`; services import only from `data/` and Firebase packages. Running an import-direction check manually against the codebase confirms no widget file imports a screen, and no model file imports `package:flutter/material.dart` beyond the minimum needed for `Gradient/IconData/Color` value types the design system uses. Each

screen file is self-contained: its private helper widgets (prefixed with an underscore, e.g. `_PostingCard`, `_ScreeningRow`) live in the same file as the screen that uses them, and are only promoted to `lib/widgets/` once a second, unrelated screen needs the same thing — avoiding the premature-abstraction failure mode of extracting a shared component after a single use.

30. Component Reusability

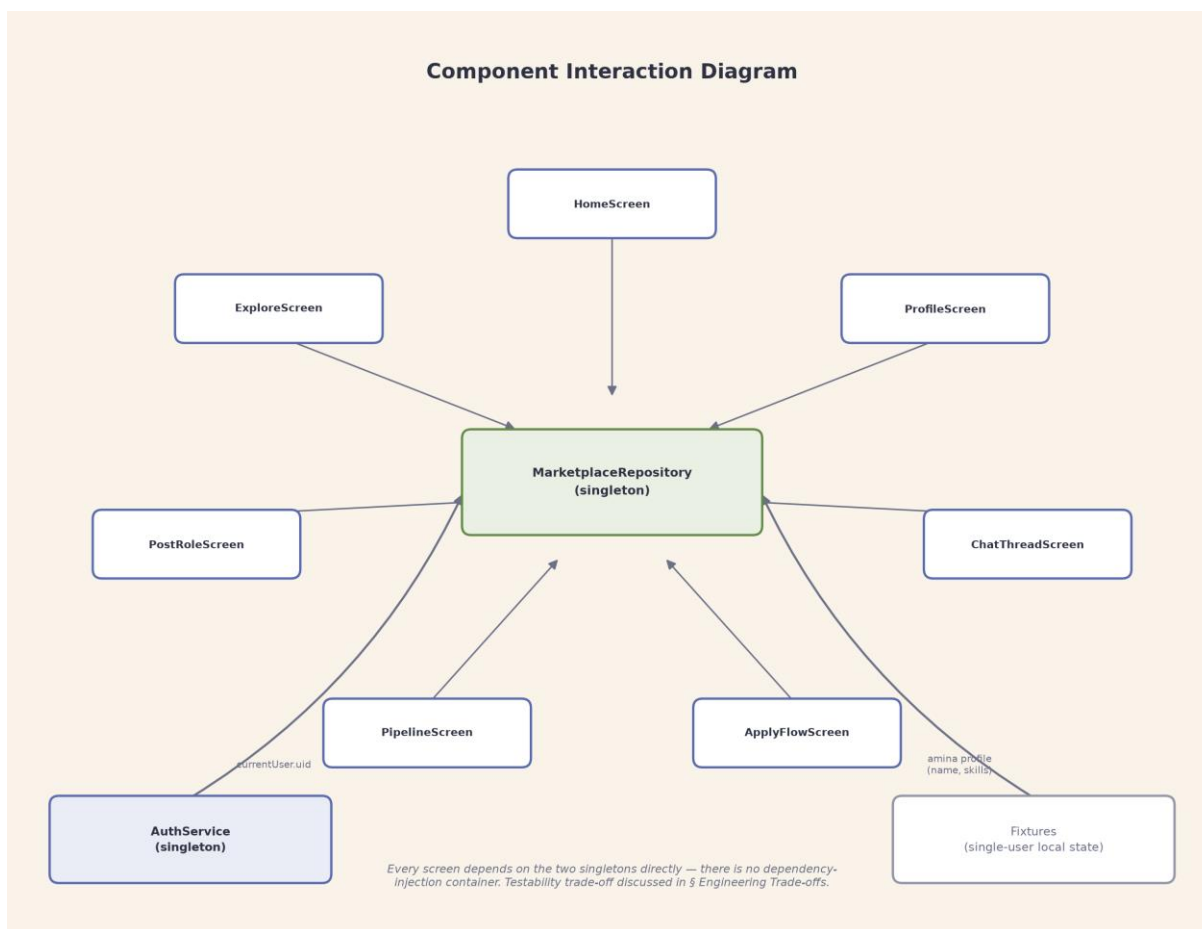


Figure 12. Every screen depends on the same two repository singletons and shared widget library.

18 widgets in `lib/widgets/` are shared across all 58 screens. A representative sample:

Widget	Used By (examples)	Reused Because
PrimaryButton / BrandButton / GradientButton	Every screen with a CTA	One place to change the pill shape, shadow, and pressed-state color for the whole app
OpportunityListCard / OpportunityHeroCard	Home, Explore, Saved	Identical role-card presentation regardless of which list is rendering it

StatusBadge / VerifiedBadge	Applications, Applicant cards, chat headers	Guarantees the tint-plus-label rule from § 19.1 can never be broken by a one-off screen
EmptyStateView	Home, Explore, Saved, Notifications, Messages, Postings	One illustration recipe (circle + icon + sparkle) instead of a bespoke empty state per screen
ChatThreadView	ChatThreadScreen (student), StartupChatThreadScreen (founder)	The message list, bubble rendering, and input bar are identical for both roles; only the header differs
LogoTile / InitialsAvatar	Every card and header showing a startup or a student	Renders a real uploaded photo/logo when present, falling back to a deterministic gradient + initials otherwise

The ChatThreadView extraction is the clearest example of reuse driven by genuine duplication rather than speculative design: the student and founder chat screens needed identical real-time message rendering and input handling, differing only in the header content (a startup's branded LogoTile vs. a student's InitialsAvatar) and which side of the conversation is "me" for bubble alignment — exactly the kind of concrete, already-duplicated logic that justifies extracting a shared widget.

31. Performance Optimizations

- IndexedStack-based navigation shells (§ 11.3) avoid re-subscribing to Firestore streams and losing scroll position on every tab switch.
- Client-side image compression (§ 20.3) keeps every Firestore document write well under the 1 MB limit, which also keeps every downstream read of that document (e.g. every screen showing a founder's LogoTile) fast, since Firestore bills and transmits by document size.
- Denormalized reads (§ 11.4) mean every list screen is exactly one query, not one query per visible item.
- Sorting (e.g. watchMyApplications() ordering by real appliedAt timestamps) is done once in the repository layer after the snapshot arrives, rather than being recomputed per rebuild inside a widget's build() method.
- shimmer-based skeletons and cached widget subtrees (const constructors used throughout the design-system widgets) keep first-paint and rebuild cost low without a profiling pass being required to notice it.

32. Engineering Trade-offs

Decision	What Was Gained	What Was Given Up
No Provider/BLoC/Riverpod (§ 12)	No indirection between a screen and the data it renders; Firestore streams already are the reactive layer	No dependency-injection seam for tests — every screen talks to the real singleton, so widget tests would need a real or faked Firestore
Repository singletons, not injected	Trivial object graph; identical to how the Firebase SDKs themselves are consumed	A screen cannot be unit-tested against a mock repository without refactoring the constructor to accept one
No Firebase Storage — logos as base64 (§ 20.3)	One fewer Firebase product to provision, configure, and secure	A hard ceiling on asset size, and no CDN-backed image delivery for a growing catalog of startup logos
Imperative Navigator, not go_router everywhere	Zero indirection for a two-shell, mobile-first app	No deep-linking or URL-addressable screens if the app ever grows a genuine web-marketing surface
Denormalized documents (§ 11.4)	Every list screen is one query	A student's profile edit (e.g. changing their name) does not retroactively update studentName on their past applications — a known staleness trade-off

33. Challenges Encountered

This project's most instructive challenges were not features that were hard to build, but real defects that were only found by treating "does the button actually do the real thing" as a first-class acceptance criterion, distinct from "does the button exist and look right." The recurring pattern: a screen built early against Fixtures (in-memory sample data) continued to look completely correct even after the backing data source was removed, because an empty list and a not-yet-loaded list render identically unless a screen explicitly checks for the difference.

Challenge	Symptom	Root Cause
Missing Firestore rules	Generic "Couldn't publish" toast on every role posting attempt	No allow rule existed for startups or notifications, so a defensive read inside postOpportunity() was silently denied (§ 24.1)
Oversized logo upload	Same generic failure, only when a real (uncompressed) photo was picked as a logo	Firestore's 1 MB document limit, hit by an unresized image_picker result on Web (§ 20.3)

Mislabeled pipeline action	A "Shortlist" button that actually extended a real offer; a ✕ that looked like "reject" but did nothing	UI copy had drifted from the method it called (§ 20.1)
Permanently "pending" outcome screen	A resolved application still displayed "Usually within 2 days" regardless of real status	The Decision timeline row was hard-coded, never reading application.status (§ 20.2)
PDF export corruption	Passport PDF text after an en-dash or curly quote rendered garbled	The bundled base-14 Helvetica font used by the pdf package cannot render those glyphs and does not fail gracefully — it corrupts the rest of the page layout instead
Spurious onboarding carousel jump	The onboarding PageView appeared to auto-advance to slide 2 with zero user input, on Windows desktop only	A transient window resize/relayout right after the screen mounts made PageView briefly report a page-change event

34. How Challenges Were Solved

- Missing Firestore rules: added explicit allow blocks for startups (public read, owner-only write) and users/{uid}/notifications (owner-only read, any-signed-in-user create, since the sender is not the recipient) — and replaced the catch(_) that had hidden the real exception with one that logs it, so the next gap like this is diagnosable in minutes, not hours.
- Oversized logo upload: replaced reliance on image_picker's unreliable resize parameters with an explicit decode → resize-to-256px → re-encode-as-JPEG pipeline using the pure-Dart image package, plus a hard 6 MB pre-check before attempting the (CPU-bound, unaccelerated-on-Web) decode at all.
- Mislabeled pipeline action: renamed the button to describe its real effect ("Extend Offer"), added a named confirmation dialog to both the offer and decline paths, and added a real declineApplicant() repository method so the ✕ button finally does something.
- Permanently pending outcome: added isAccepted/isDeclined/isResolved checks that drive a real celebration or decline banner and a real timeline state, and hid the now-irrelevant Withdraw action once resolved.
- PDF corruption: added a _safe() sanitizer that transliterates en/em dashes, curly quotes, and ellipses to their plain-ASCII equivalents before any dynamic string reaches a pw.Text widget.
- Onboarding carousel: added a 500ms "settling" window after the screen mounts during which any reported page change is treated as spurious and the PageView is snapped back to page 0.

35. Lessons Learned

- A generic catch (`_`) block is a debugging tax paid later with interest — every exception handler in this codebase now at minimum `debugPrints` the real error, even when the user-facing message stays simple.
- Fixture/sample data is a liability the moment a real backend exists alongside it: a screen that looks correct against `Fixtures.opportunities` can hide a completely disconnected real data path for a long time, because both an empty real stream and a populated fixture list render as "a screen with content."
- A button's label is a promise; auditing screens by clicking through every action (not just reading the code) is the only way to catch the specific case where a label and its handler have quietly diverged.
- Third-party font/rendering limitations (the Helvetica dash bug) do not announce themselves as errors — they silently corrupt output, which means defensive sanitization of dynamic strings is cheaper than hoping input never contains an unusual character.

36. Testing Strategy

Automated coverage is intentionally the area this report is most direct about: `test/widget_test.dart` contains Flutter's default counter-app smoke test scaffold and has not been extended with feature-level widget or integration tests. Verification during development instead relied on flutter analyze as a continuous static-analysis gate (kept clean after every change across the project's history) plus manual, scripted click-through testing of every screen's every interactive element — the same technique that surfaced every defect catalogued in § 33. One genuine automated test was written ad hoc to debug the PDF font issue (§ 33): a flutter test using `tester.runAsync()` to call `RenderRepaintBoundary.toImage()` and visually confirm the rendered PDF page, which is recorded here because `runAsync()` is required — the call hangs forever inside a standard widget test otherwise.

36.1 Why Static Analysis Was Treated as a Hard Gate

flutter analyze catches an entire class of defect for zero ongoing cost: unused imports and fields left behind after a refactor, undefined names after a rename, and (via `flutter_lints`) a large set of Dart idiom violations. Every code change in this project's history was required to leave flutter analyze at "No issues found!" before being considered complete, which caught several of the smaller mistakes made while wiring up the fixes in § 34 (mismatched parameter names, unused fields left after a partial edit) before they ever reached manual testing.

37. Debugging Strategy

The debugging approach that resolved every defect in § 33 followed the same three steps: (1) reproduce the symptom exactly as reported, (2) replace any exception handler that could be hiding the real cause with one that surfaces it (debugPrint, or — temporarily, during live diagnosis — the raw exception text inside the user-facing toast itself), and (3) trace the surfaced error back to its root cause in the repository or rules layer rather than patching the symptom at the screen level. The Firestore rules gap (§ 24.1) is the clearest example: the visible symptom ("Couldn't publish") was three layers away from its actual cause (a missing allow rule on an unrelated collection), and would not have been found without deliberately surfacing the swallowed FirebaseException.

38. Limitations

Stated plainly, without euphemism:

- No automated feature-level test suite exists (§ 36) — regressions are currently caught by manual click-through testing and flutter analyze, not by CI-enforced tests.
- A student's own profile (StudentProfile, held in Fixtures.amina) is not Firestore-backed at all — it is a single in-memory object for the current app session, meaning it does not persist across app restarts and would not support more than one concurrent student identity on one device.
- Firebase Storage is not provisioned, so all media (startup logos) must fit inside a Firestore document as base64 text, capping practical image size well below what a dedicated object store would allow.
- No push notifications (Cloud Messaging) exist — the in-app Notifications screen only updates while the app is open and subscribed to its Firestore stream.
- No true responsive breakpoint system (§ 22) — the layout is tuned for a single mobile-portrait width.
- Analytics (§ funnel screen) is computed entirely from real applicant documents at read time; page-view or traffic analytics would require dedicated event logging that does not exist yet.
- Declined and closed applications are filtered out of the founder's active Kanban pipeline entirely rather than archived into a dedicated view — a deliberate scope decision (§ 20.1) that has no "see everyone I've ever declined" screen as a consequence.

39. Future Improvements

Improvement	Why It's Next	Rough Shape of the Change
Persist StudentProfile to Firestore	The single biggest remaining gap between the founder side (fully real) and student side (profile still local)	A students/{uid} collection mirroring startups/{uid}, with the same save/watch pattern already proven in

		MarketplaceRepository
Automated widget/integration tests	Regressions currently rely entirely on manual testing	Fake/in-memory implementations of AuthService and MarketplaceRepository behind an interface, injected via constructor rather than accessed as a bare singleton
Firebase Cloud Messaging	Notifications currently require the app to be open	A Cloud Function triggered on notification-document creation, forwarding to FCM
Founder-side declined/archive view	Founders currently lose visibility into who they've declined	A filter tab on Pipeline reading the same applications collection by status
Pagination on live opportunity feeds	Needed once the catalog exceeds a few hundred concurrent postings (§ 23)	Firestore cursor-based pagination (startAfter) added to watchLiveOpportunities()
Real Semantics annotations on custom widgets	Screen-reader support currently relies entirely on inherited Material widget semantics	Explicit Semantics(label: ...) wrapping for the bespoke cards/badges in lib/widgets/

40. Product Vision

ALULink's long-term value is not the marketplace mechanics themselves — job boards are a commodity — it is the Experience Passport becoming the trusted, portable record of what a student actually did, verified by name against a real founder's real, verified startup. A campus recruiter, a graduate programme, or a future employer should eventually be able to trust a Passport entry the way a reference letter is trusted today, but structured, timestamped, and impossible to quietly embellish. Every product decision in this report that looks like it is only about "making the demo honest" — real offer/decline outcomes, a founder who cannot silently mislabel an action, screening answers tied to the question that was actually asked — is in service of that same end goal: a system whose data can be trusted enough to matter outside the app itself. The natural next step toward that vision is the Passport becoming shareable via a public, verifiable link, and screening/interview data becoming rich enough that a founder's endorsement carries real signal rather than being a rubber stamp.

41. Conclusion

ALULink demonstrates a complete, Firebase-backed, two-sided marketplace built on a single Flutter codebase, with every core workflow — authentication, posting, applying, screening, pipeline management, messaging, scheduling, offer resolution, and verified experience export — backed by real Cloud Firestore data rather than mock content. Its architecture makes a small number of deliberate,

documented bets (no external state-management package, no custom backend server, denormalized documents over joins) that are appropriate to its actual scale and requirements, and every one of those bets is recorded here alongside the trade-off it accepts. Where real defects were found — a mislabeled action, a missing security rule, an oversized upload, a permanently-pending status screen — they are documented as concretely as the features are, because a report that only describes what works is not a trustworthy account of an engineering project, and trustworthiness is the entire point of the product it describes.